Activity 33

Observe Behavior

Add instrumentation to the software system so you can see runtime behaviors firsthand. Use the observations to answer specific questions about quality attributes and other stakeholder concerns. Once instrumentation is in place, either observe the system in normal use or inject stimuli to flex specific quality attribute scenarios.

Observing behavior is a great way to analyze runtime quality attributes. The ability to observe the system assumes that *observability* is designed into the architecture. Evolve the architecture as needed to promote required observability scenarios.

Benefits

- Monitor the system over time to verify design assumptions.
- Directly test how well quality attributes are promoted.
- Produce concrete metrics that can be shared with stakeholders.

Activity Timing

Varies depending on the required analysis and how well the software system promotes observability.

Participants

One or more analysts, usually developers of the system.

Preparation and Materials

- To add instrumentation there must be a working (or partially working) software system. Adding instrumentation can sometimes be a design task unto itself. Decisions around frameworks, data storage, and analysis must be made before observations can begin.

Steps

1. Define the goals of the analysis. What question are you trying to answer? Use Activity 3, *Goal-Question-Metric (GQM) Workshop*, on page 199 to identify candidate metrics and the data required to compute those metrics.
2. Decide how to generate data and design tests to drive the system.
3. Add the required instrumentation and logging to the software system. Verify that your changes work before attempting meaningful analysis. You don't want to spend a week running tests only to learn that your logging failed!
4. Implement and execute tests, or allow the software system to be used as it normally would.
5. Once data has been collected, perform the analysis. Compute metrics and answer the questions established in step 1. If you are unable to answer questions, then make adjustments and try again.
6. Prepare and share findings with relevant stakeholders.

Guidelines and Hints

- Observability is a quality attribute and must be designed into the architecture. Instrumentation can be added late, even after the system is in the wild, as long as you've designed the ability to produce and collect system events into the architecture.
- As you answer questions about the software system, think about how the data can be used in automated analysis. Consider adding your metrics to system dashboards and alerting systems.
- In theory, any runtime property of the system can be observed, including security, performance, availability, and reliability, among others.

Example

Some patterns such as event sourcing publish-subscribe (described on page 88) have observability baked in. Observability is a must-have for all modern distributed systems, especially microservices.

Netflix has done extensive work in this area and made much of their work available as open source.[4] One example is the Hystrix Dashboard, which allows developers to observe metrics produced by the Hystrix fault tolerance library for the JVM.[5] Another example is the Simian Army, a suite of tools used to stimulate a service-oriented system in various ways.[6]

In the simplest case, you can use any logging platform to record observed information. Take a look at logging platforms such as LogStash,[7] Splunk,[8] or Graylog[9] for storing, visualizing, and analyzing system events. Keep in mind that though these tools are powerful, their effectiveness depends heavily on how you've instrumented the system.

4. https://netflix.github.io/
5. https://github.com/Netflix/Hystrix
6. https://github.com/Netflix/SimianArmy
7. https://www.elastic.co/products/logstash
8. https://www.splunk.com/
9. https://www.graylog.org

Activity 34

Question–Comment–Concern

This is a collaborative, visual activity that gets the whole team talking about the architecture—what they know, what they don't know, and what keeps them up at night. Use this activity to shine a light on knowledge gaps, articulate issues, and establish known facts about the architecture.

The inclusion of comments during the workshop is meant to fast-track issue resolution. Issues that result from simple gaps in understanding can be resolved immediately so the team can focus on the bigger concerns. As an architect you can even choose to add questions during the workshop as a simple sanity check (described on page 304).

Benefits

- Promote knowledge sharing by flushing questions into the open along with potential answers.
- Visualize high-risk, mysterious, or troublesome parts of the system.
- Identify areas in need of further research and exploration.
- Foster shared ownership over the architecture and the direction it takes.

Activity Timing

30–90 minutes

Participants

Whole team, about 3–7 participants

Preparation and Materials

- Views of the architecture. These can be created just-in-time or printed from appropriate sources.
- Sticky notes (three different colors), markers
- Large paper, flipcharts, or whiteboard with appropriate markers

Steps

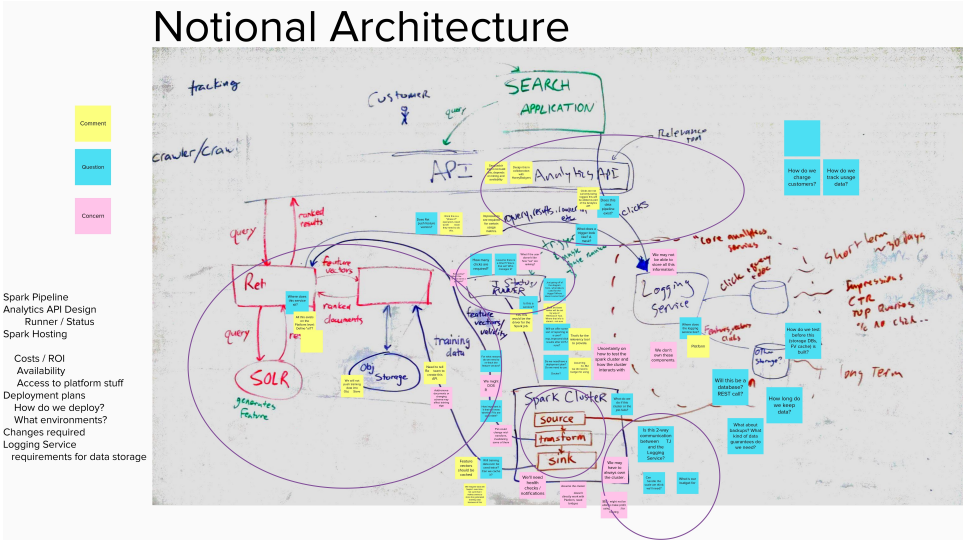
1. Start the workshop by explaining the goals. For example, *In this workshop we will learn what we know, what we think we know, and what worries us about the architecture.*
2. Sketch relevant views. All participants should help sketch views of the architecture on whiteboards or paper.
3. Brainstorm questions, comments, and concerns. Working together, participants will write one item per sticky note and immediately place them on the relevant view.
4. Stop the exercise when time expires or as the rate of stickies slows.
5. Observe and reflect. What does the team notice about the diagram? What is interesting about where sticky notes landed? Are there areas of particularly high concern or uncertainty? Were there any areas with many questions that were quickly answered?
6. Extract themes. Read through the sticky notes and write down common themes that emerge.
7. Decide on next steps. Briefly brainstorm actions that should be taken next. Prioritize and assign responsibility for next steps.

Guidelines and Hints

- Assign a color to each type of item before brainstorming starts. Create a legend that is visible to all participants.
- Comments can be facts, new ideas, tidbits of knowledge, or answers to questions raised during the workshop. To answer a question, simply write the answer on a comment sticky note and place it directly on top of the question it answers.
- Concerns can be known problems, risks, or general worries.
- Pay attention to questions. Questions can expose gaps in understanding, mismatches in expectations, or areas in need of further exploration.

Example

In this example, the team sketched diagrams during a whiteboard jam (see on page 255), took a picture with a phone, and uploaded the image to Mural[10] to run the exercise. Notice how comments are placed in close proximity to the questions they answer.



10. https://mural.co/

Activity 35

Risk Storming

A collaborative, visual technique for identifying risks in the architecture. Risk storming was proposed by Simon Brown in *Software Architecture for Developers [Bro16]*.

Benefits

- Quickly identify risks in the proposed system architecture.
- Visualize the system by considering the level of risk.
- Constrain risk identification to architectural concerns.
- Provide a platform for all team members to elevate their concerns.

Activity Timing

60–90 minutes.

Participants

Small groups of 3–7 developers. Participants must be familiar with the architecture. This workshop can be self-facilitated by experienced participants.

Preparation and Materials

- Views of the architecture. These can be created just-in-time or printed from appropriate sources.
- Sticky notes (three different colors), markers
- Large paper, flipcharts, or whiteboard with appropriate markers

Steps

1. Set the expectations for the workshop by explaining the workshop goals. For example, *By the end of this workshop we will have a list of prioritized risks to help us decide on next steps.*
2. Sketch relevant views. All participants should help sketch views of the architecture on whiteboards or paper. Include a range of views.

3. Brainstorm risks. Working individually, participants will write one risk per sticky note. The color of the sticky note used to capture the risk should correspond to the risk's degree of exposure: high, medium, or low. Exposure is a relative qualification of how bad a risk is by considering probability, impact, and time frame.
4. Cluster risks on the diagrams. Participants place their risks on the diagrams where they think the risk most directly applies.
5. Prioritize and discuss the identified risks. Look at clusters of sticky notes, high-exposure risks, or other interesting patterns.
6. Develop mitigation strategies and decide on next steps.

Guidelines and Hints

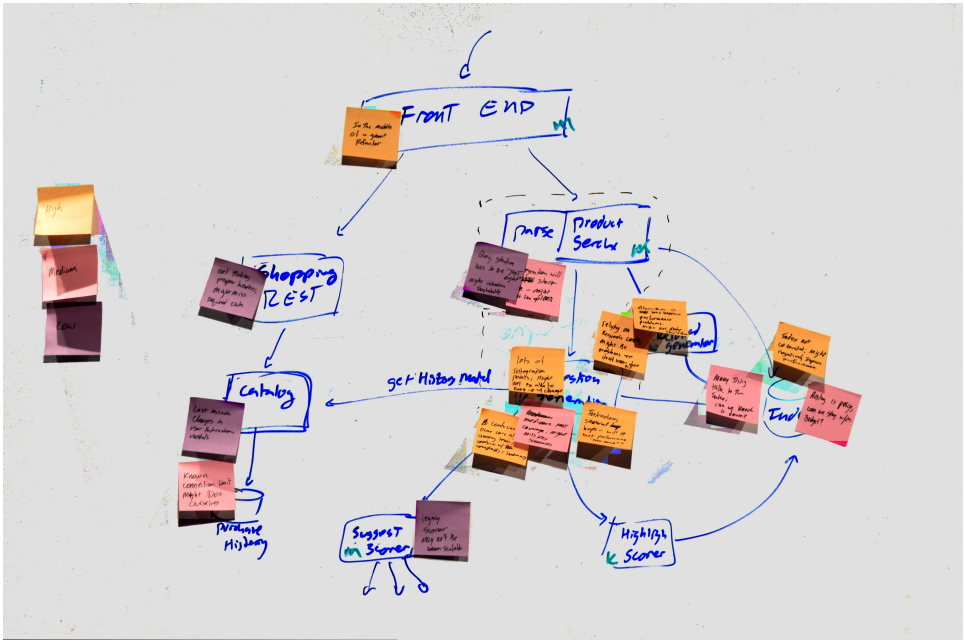
- Place sticky notes directly on the diagrams.
- Stick duplicate risks directly on top of each other.
- Leave time for team discussions. This is the most important part.
- Use no more than 2–3 sketches. More than that can be overwhelming.

Example

Here is a tentative agenda for the workshop:

Introduction and goals	< 5 minutes
Sketching	15–20 minutes
Risk brainstorming	7–15 minutes
Discussion and prioritizing	15–30 minutes
Brainstorm mitigations for top risks	10–15 minutes
Wrap-up, review actions	< 5 minutes

In this example, a sticky note legend is posted on the left side. High-exposure risks are orange, medium risks are pink, and low-exposure risks are purple. It's easy to see clusters of high-exposure risks around certain elements and relations. These areas should receive more design attention. Likewise, there appear to be areas of low risk that may already be in development or could get fully underway soon.



Activity 36

Sanity Check

A fast, simple exercise designed to expose issues in team communication or understanding. Sanity checks verify everyone is indeed on the same page. They can also identify opportunities for improving team operations, artifacts, and design methods.

When designing a sanity check, think back to the pop quizzes you dreaded in elementary school. Any short-burst activity that forces teammates to actively think about the architecture is a great sanity check candidate. Verbal sanity checks are an efficient way to end most collaborative design sessions.

Benefits

- Reinforce architectural responsibility across the team.
- Identify problems caused by misunderstandings and gaps in knowledge early.
- Create teaching and coaching opportunities.
- Document knowledge the team feels is essential to the design.
- Pinpoint opportunities for improving artifacts and communication.
- Uncover the unknown unknowns.

Activity Timing

5–10 minutes

Participants

Whole team, assign one team member to prepare the sanity check.

Preparation and Materials

- A prepared exercise or quiz.

Steps

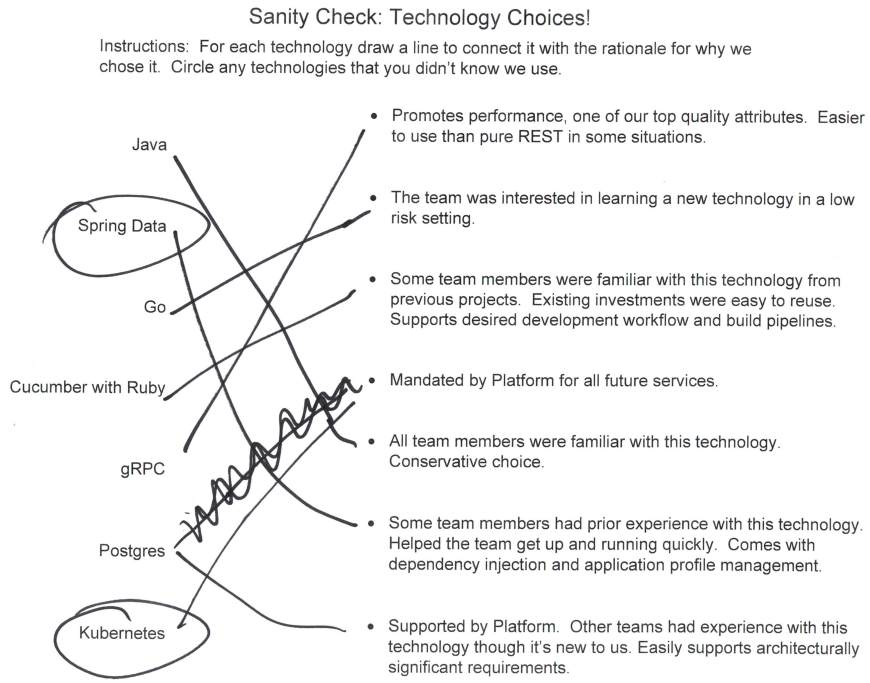
1. Remind the team that sanity checks are for process improvement and helping uncover knowledge gaps before they become problems.
2. Complete the exercise.
3. Review answers to the exercise. Briefly discuss answers that differed from the key.
4. Decide whether follow-up actions are required. If so, the person who led the sanity check will lead the team in determining next steps.

Guidelines and Hints

- Keep it simple and short. A good sanity check can be completed in 5 minutes or less.
- Always remember: Sanity checks are really about improving the team's knowledge. Do not use sanity checks to punish or promote teammates.
- Share responsibility for creating sanity checks among the team.
- Host regular sanity checks—for example, at the beginning or end of a weekly status meeting or retrospective.
- Get creative and use a variety of formats. This keeps the sanity checks fresh and helps expose different kinds of understanding gaps.

Example

Sanity checks can take many forms. Some examples include true/false, fill in the blank, matching, and multiple choice. And this is just the beginning. In the following example sanity check, the team verified the rationale for specific technology choices by playing a simple matching game.



Activity 37

Scenario Walkthrough

Describe step-by-step how the architecture addresses a specific quality attribute scenario. Scenario walkthroughs can be used any time but are most applicable early in the life of the software system, before the system's behavior can be observed directly.

A scenario walkthrough is like telling a story about the architecture. Pick a quality attribute scenario and describe what the system would do in response to the scenario stimulus. As you walk through the various elements in your design, show how the quality attribute is promoted (or not) by the system.

Benefits

- Assess the architecture design early, even while it's only on paper.
- Identify different concerns in the architecture.
- Reason about how the architecture will respond to different stimuli.
- Qualify the design. Walkthroughs are not strict pass or fail.
- Quickly determine the extent to which the architecture promotes or inhibits different quality attributes.

Activity Timing

Walking through the architecture for a single quality attribute scenario might take 20–30 minutes depending on the system. A scenario walkthrough meeting will typically run 1–3 hours and cover several quality attribute scenarios.

Participants

Scenarios walkthroughs require that the following roles are filled:

- The *architect* is someone knowledgeable in the architecture's design. This person describes how the system responds to stimuli.
- The *recorder* take notes during the meeting. This person will write down any issues, risks, unknowns, gaps, and other general concerns raised during the session.

- The *reader* reads scenarios to start the discussion and facilitates the walkthrough. This person is also the session moderator.
- Every walkthrough will have one or more *reviewers*. Reviewers are relevant stakeholders or knowledgeable non-stakeholders who can ask questions and poke holes in the architecture during the review. On small teams, the recorder and reader might also be reviewers.

Walkthroughs should be small, with no more than 3–7 participants.

Preparation and Materials

- Reviewers should look at the quality attribute scenarios, relevant architecture descriptions, and other background materials as homework prior to the review meeting. If such materials do not exist, prepare an introductory presentation (such as the architecture briefing described on page 286) and allow extra time to bring reviewers up to speed.
- Prioritized quality attribute scenarios must be prepared prior to the start of the meeting.

Steps

1. Distribute scenarios and architecture artifacts to the group. Set up projectors or screen sharing so the architect can easily share relevant views.
2. The reader picks a quality attribute scenario and reads it out loud. The purpose of reading the whole scenario is to ensure everyone knows the context and general scope of the scenario.
3. The reader then repeats the stimulus of the quality attribute scenario, thus kicking off the walkthrough.
4. The architect describes how the system responds to the stimulus, by *walking through* elements in the architecture.
5. Once the architect has completed the initial walkthrough, reviewers may ask questions and point out potential architectural issues.
6. The architect may briefly respond to questions. Issues, risks, and questions raised should be recorded for further analysis by the team after the review meeting.
7. After all reviewers have shared their feedback or time has expired, pick another scenario and repeat the process.

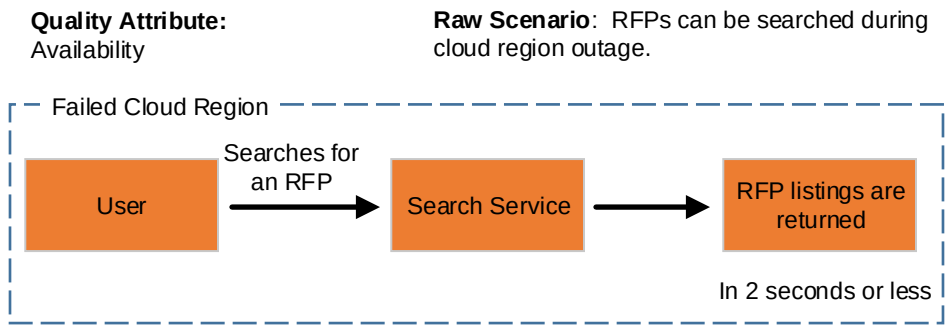
Guidelines and Hints

- Avoid turning the review into a witch hunt. The whole point is to find potential problems while they are still cheap to fix—on paper. Finding issues is a good thing.
- To keep the meeting moving, the time spent on each scenario should be time boxed.
- Avoid problem solving during the review. The purpose of this activity is to surface issues, not solve them.
- The reader and recorder roles can be combined but should be separate from the architect role.
- Rotate roles to help build teammates' skills.
- Write or project the current the quality attribute scenario so reviewers can see it during the walkthrough. If this is not practical, distribute a packet with quality attribute scenarios.
- Record new quality attribute scenarios raised during the walkthrough.

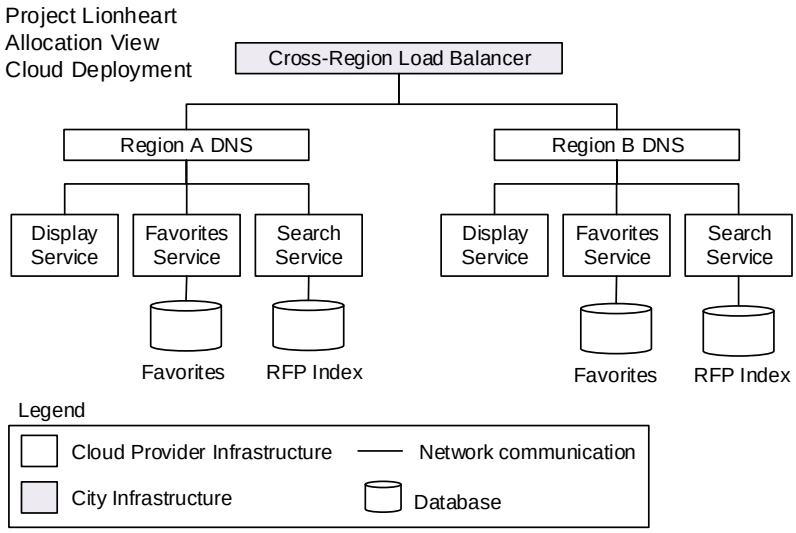
Example

Let's walk through an availability scenario from Project Lionheart. A user's searches for open RFPs and receives a list of available RFPs 99 percent of the time on average over the course of the year. To walk through an availability scenario we'll need to focus on specific conditions. Recall that Project Lionheart consists of a small handful of web services, a few databases, and a search index.

Here's a specific quality attribute scenario:



The City of Springfield's IT Department has decided to host Project Lionheart services using a popular cloud provider. Services and processes are hosted in Docker containers in two different cloud regions. This way, when one region fails another is still available.



Here is one potential walkthrough for the given quality attribute scenario:

> ***Reader:*** *The next scenario covers availability during a region failure. Let's start by assuming everything is up and running. OK, bam! Region A just went down.*
>
> ***Architect:*** *Within 60 seconds, our cross-region load balancer will detect the failure and automatically route traffic to the available region. If a user was unlucky enough to be on the website at that moment, they'll get a failure. Refreshing the browser should fix the problem.*
>
> ***Reviewer 1:*** *Where is the multiregion load balancer hosted?*
>
> ***Architect:*** *In the closet across the hall.*
>
> ***Reviewer 1:*** *So all site accessibility is determined by load balancers we're managing? Why bother with the cloud platform at all if our weakest link is in the building?*
>
> ***Reader:*** *Reviewer 1, let's work to keep our conversation constructive. Can you try to phrase your concern as a risk?*
>
> ***Reviewer 1:*** *Sorry, I was just a bit surprised. How about this: There is a single point-of-failure in our cross-region strategy; might not be able to meet required service-level agreements. (Recorder verifies the concern is captured.)*
>
> ***Reviewer 2:*** *How is the data kept up to date in the proposed multiregion deployment…*

Activity 38

Sketch and Compare

Sometimes a design only seems good because there isn't a baseline for comparison. With the sketch and compare activity, we create two or more alternatives of the same design so it's easier to see the pros and cons.

Any design alternatives can be sketched and compared. This includes current and future, ideal and reality, technology A and technology B, and many others. Sketching the extremes can also be compared. To do this, pick a quality attribute or design concept and design the architecture for that one thing at the exclusion of all else. Then pick another high-priority quality attribute or interesting design concept and sketch an alternative design for comparison.

Benefits

- Expose both the positive and negative aspects of a decision by comparing it to something else.
- Create a platform for discussion and build consensus around a design decision.
- Avoid *buyer's remorse* when making a design decision by doing at least a basic comparison.

Activity Timing

20–30 minutes, up to an hour

Participants

This activity works best in small groups of 3–5 people, including the architect and other stakeholders.

Preparation and Materials

- Whiteboard or flipchart, markers
- Alternatively, prepared views (for example, in slide software), projector

Steps

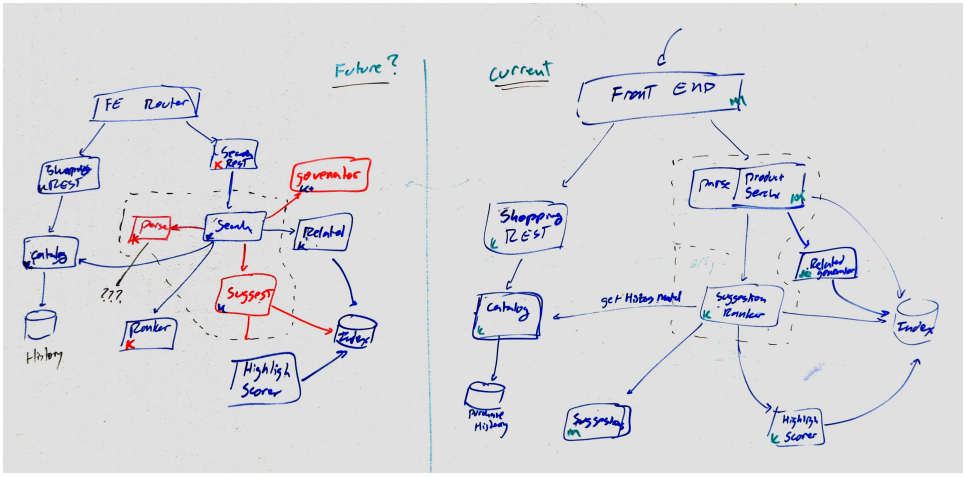
1. Establish the goals of the activity by saying something like, *It seems like there are at least two alternative designs on the table. Let's put them side by side and pick one.*
2. Sketch or show the alternative designs so everyone can see them.
3. Open the discussion by pointing out an advantage or disadvantage of one design compared to the other. Invite others to share their thoughts.
4. Write down participants' ideas as they share them. If necessary, change or annotate the diagrams to clarify meaning or add new insights.
5. As the group begins to reach consensus, summarize the decisions made. Give a just-in-time sanity check (introduced on page 304) to verify that participants understand and agree with the decisions.
6. Take pictures, record the decisions in your team wiki, and use the discussion to help flesh out design rationale.

Guidelines and Hints

- Help participants avoid argumentative confrontations and encourage constructive participation. Sometimes participants will pick sides and entrench themselves, strongly favoring one design over the other.
- Be ready to sketch compromises as the discussion progresses. Some of these new ideas will become new design alternatives.
- Always summarize the findings. Skipping the final summary can leave participants confused about the decisions made.
- Follow up with skeptics to win them over and drive consensus.

Example

In this example, the team was struggling with a few ideas. Parts of the software system were undergoing major refactoring and the team had a strong desire to avoid rework if possible. Several new responsibilities needed a home and two alternatives had been proposed. A deadline was looming and a decision had to be made with imperfect information.



In this case, the team sketched diagrams of the *current* and *possible future* designs. Some team members were not satisfied with the current architecture. After seeing how it compared to the potential future state, they were much more willing to go along with the current design. The discussion that went with these sketches also created a shared understanding for the future direction of the system and identified areas with potential technical debt.

APPENDIX 1

Community Contributor Bios

Len Bass is the coauthor of two award-winning books in software architecture: *Software Architecture in Practice [BCK12]* and *Documenting Software Architectures: Views and Beyond [BBCG10]*, as well as several other books and numerous papers in computer science and software engineering, including his latest book, *DevOps: A Software Architect's Perspective [BWZ15]*. Len has over 50 years' experience in software development, 25 of those at the Software Engineering Institute. He also worked for three years at National ICT Australia Ltd. (NICTA) and is currently an adjunct faculty at Carnegie Mellon.

Bett Bollhoefer has worked in the software space since 1999. Today, Bett is an architect at GE Digital focused on Predix Industrial Internet of Things Platform Architecture. Before joining GE, she first worked as a developer, then as a solutions architect at Verizon. Bett speaks and writes on software design and is the author of several books, including *You Can Be a Software Architect [Cor13]* and *The Zen of Software Development: A Seven Day Journey: A Handbook to Enlightened Software Development [Cor15]*. Bett cohosted the popular Software Architecture Concepts podcast for two years.[1] She is a Distinguished Toastmaster, former president of Distinguished Division Governor in Toastmasters, and winner of the Division Governor of the Year award. And for fun, Bett is a professional improv actor, and enjoys swing dancing, painting, and playing the cello.

Simon Brown is an independent consultant specializing in software architecture and the author of *Software Architecture for Developers [Bro16]*, a developer-friendly guide to software architecture, technical leadership and the balance with agility. He is also the creator of the C4 software

1. http://www.architecturecast.net/

architecture model and Structurizr,[2] which is a collection of tooling to help software teams visualize, document, and explore their software architecture. Follow him on Twitter @simonbrown[3] or his website, http://www.simonbrown.je.

George Fairbanks has been teaching software architecture and design since 1998, is the author of the book *Just Enough Software Architecture: A Risk-Driven Approach [Fai10]*, has a PhD in software engineering from Carnegie Mellon University, and is a software engineer at Google.

Thijmen de Gooijer brings business and software together through architecture, putting the customer and quality first. Thijmen co-authored over ten research publications on architecture, while collaborating with other architecture and user experience researchers throughout Europe, India, and the United States. He graduated cum laude in software engineering with a double MSc degree from VU University in Amsterdam, the Netherlands and Malardalen University in Västerås, Sweden.

Patrick Kua is a principal technical consultant for ThoughtWorks in London and the author of two books: *The Retrospective Handbook: A Guide for Agile Teams [Kua13]* and *Talking with Tech Leads: From Novices to Practitioners [Kua15]*. Patrick is a frequent conference speaker and blogger who is passionate about bringing a balanced focused between people, organizations. and technology. Follow him on Twitter @patkua[4] or his website, https://www.thekua.com/atwork.

Ipek Ozkaya is a senior member of the technical staff at the Carnegie Mellon University (CMU) Software Engineering Institute (SEI). Her primary interests include developing techniques for improving software development efficiency and system evolution with an emphasis on software architecture practices, software economics, and agile development. Her most recent work focuses on building the theoretical and empirical foundations of managing technical debt in large-scale, complex software-intensive systems. Ozkaya serves on the advisory and editorial boards of *IEEE Software Magazine* and as an adjunct faculty member for the Master of Software Engineering Program at CMU. She earned a PhD in computational design from CMU. Follow her on Twitter @ipekozkaya.[5]

2. https://structurizr.com
3. https://twitter.com/simonbrown
4. https://twitter.com/patkua
5. https://twitter.com/ipekozkaya

Bibliography

[AISJ77] Christopher Alexander, Sara Ishikawa, Murray Silverstein, Max Jacobson, Ingrid Fiksdahl-King, and Shlomo Angel. *A Pattern Language: Towns, Buildings, Construction*. Oxford University Press, New York, NY, 1977.

[Ale64] Christopher Alexander. *Notes on the Synthesis of Form*. Harvard University Press, Boston, MA, 1964.

[Amb04] Scott Ambler. *The Object Primer: Agile Model-Driven Development with UML 2.0*. Cambridge University Press, Cambridge, United Kingdom, Third edition, 2004.

[App11] Juregen Appelo. *Management 3.0: Leading Agile Developers, Developing Agile Leaders*. Addison-Wesley, Boston, MA, 2011.

[App16] Juregen Appelo. *Managing for Happiness: Games, Tools, and Practices to Motivate Any Team*. John Wiley & Sons, New York, NY, 2016.

[BBCG10] Felix Bachmann, Len Bass, Paul C. Clements, David Garlan, James Ivers, Reed Little, Paulo Merson, Robert Nord, and Judith A. Stafford. *Documenting Software Architectures: Views and Beyond*. Addison-Wesley, Boston, MA, Second edition, 2010.

[BC89] Kent Beck and Ward Cunningham. A Laboratory for Teaching Object-Oriented Thinking. *ACM SIGPLAN Notices*. 24[10], 1989, October.

[BCK12] Len Bass, Paul C. Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, Boston, MA, Third edition, 2012.

[BCR94] Victor Basili, Gianluigi Caldiera, and H. Dieter Rombach. The Goal Question Metric (GQM) Approach. *Encyclopedia of Software Engineering 1*. 528–532, 1994.

[Bec00] Kent Beck. *Extreme Programming Explained: Embrace Change*. Addison-Wesley Longman, Boston, MA, 2000.

[BELS03] Mario R. Barbacci, Robert J. Ellison, Anthony J. Lattanze, Judith A. Stafford, Charles B. Weinstock, and William G. Wood. Quality Attribute Workshops (QAWs), Third edition. *Software Engineering Institute Digital Library*. 2003.

[Bra17] Alberto Brandolini. *Introducing Event Storming: An Act of Deliberate Collective Learning*. LeanPub, https://leanpub.com, 2017.

[Bro16] Simon Brown. *Software Architecture for Developers*. LeanPub, https://leanpub.com, 2016.

[Bro86] Frederick Brooks. No Silver Bullet—Essence and Accident in Software Engineering. *Proceedings of the IFIP Tenth World Computing Conference*. 1986.

[Bro95] Frederick P. Brooks Jr. *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, Boston, MA, Anniversary, 1995.

[BT03] Barry Boehm and Richard Turner. Using Risk to Balance Agile and Plan-Driven Methods. *IEEE Computer*. 36[6]:57–66, 2003, June.

[BWO10] Barry Boehm, Greg Wilson (editor), and Adam Oram (editor). *Architecting: How Much and When?*. O'Reilly & Associates, Inc., Sebastopol, CA, 2010.

[BWZ15] Len Bass, Ingo Weber, and Liming Zhu. *DevOps: A Software Architect's Perspective*. Addison-Wesley, Boston, MA, 2015.

[Car09] Dale Carnegie. *How to Win Friends and Influence People*. Simon & Schuster, New York, NY, Third edition, 2009.

[Coh09] Mike Cohn. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley, Boston, MA, 2009.

[Cor13] Bett Correa. *You Can Be a Software Architect*. CreateSpace, an Amazon Company, Seattle, WA, 2013.

[Cor15] Bett Correa. *The Zen of Software Development: A Seven Day Journey: A Handbook to Enlightened Software Development*. CreateSpace, an Amazon Company, Seattle, WA, 2015.

[Eva03] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Longman, Boston, MA, First, 2003.

[Fai10] George Fairbanks. *Just Enough Software Architecture: A Risk-Driven Approach*. Marshall & Brainerd, Boulder, CO, 2010.

[FHR99] Brian Foote, Niel Harrison, and Hans Rohnert. *Pattern Languages of Program Design 4*. Addison-Wesley, Boston, MA, 1999.

[Fow03] Martin Fowler. Who Needs an Architect?. *IEEE Software*. 20[5]:11–13, 2003, September/October.

[GAO95] David Garlan, Robert Allen, and John Ockerbloom. Architectural Mismatch: Why Reuse Is So Hard. *IEEE Software*. 12:17–26, 1995, November.

[GHJV95] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1995.

[Glu94] David P. Gluch. A Construct for Describing Software Development Risks. *Software Engineering Institute Digital Library*. 1994, July.

[Goo09] Kim Goodwin. *Designing for the Digital Age: How to Create Human-Centered Products and Services*. John Wiley & Sons, New York, NY, 2009.

[Hoh16] Gregor Hohpe. *37 Things and Architect Knows: A Chief Architect's Journey*. LeanPub, https://leanpub.com, 2016.

[HW04] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing and Deploying Messaging Solutions*. Addison-Wesley, Boston, MA, 2004.

[Int11] International Organization for Standards (ISO). ISO/IEC/IEEE 42010:2011 Systems and software engineering – Architecture description. *IEEE*. 2011, December.

[Kee15] Michael Keeling. Architecture Haiku: A Case Study in Lean Documentation. *IEEE Software*. 32[3]:35-39, 2015, May/June.

[KKC00] Rick Kazman, Mark H. Klein, and Paul C. Clements. ATAM: Method for Architecture Evaluation. *Software Engineering Institute Digital Library*. 2000.

[Kru95] Phillippe Krutchen. Architectural Blueprints — The 4+1 View Model of Software Architecture. *IEEE Software*. 12[6]:42–50, 1995.

[Kua13] Patrick Kua. *The Retrospective Handbook: A Guide for Agile Teams*. CreateSpace, an Amazon Company, Seattle, WA, 2013.

[Kua15] Patrick Kua. *Talking with Tech Leads: From Novices to Practitioners*. CreateSpace, an Amazon Company, Seattle, WA, 2015.

[KV11] Michael Keeling and Michail Velichansky. Making Metaphors That Matter. *Proceedings of the 2011 Agile Conference*. 256–262, 2011.

[MB02] Ruth Malan and Dana Bredemeyer. Less is more with minimalist architecture. *IT Professional*. 4[5]:48, 46–47, 2002, September-October.

[Mey97] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, Englewood Cliffs, NJ, Second edition, 1997.

[Mug14] Jonathan Mugan. *The Curiosity Cycle: Preparing Your Child for the Ongoing Technological Explosion*. Mugan Publishing, Buda, TX, Second edition, 2014.

[Pat14] Jeff Patton. *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly & Associates, Inc., Sebastopol, CA, 2014.

[PML10] Hasso Plattner, Chrisoph Meinel, and Larry Leifer. *Design Thinking: Understand - Improve - Apply (Understanding Innovation)*. Springer, New York, NY, 2010.

[PP03] Mary Poppendieck and Tom Poppendieck. *Lean Software Development: An Agile Toolkit for Software Development Managers*. Addison-Wesley, Boston, MA, 2003.

[Ras10] Jonathan Rasmusson. *The Agile Samurai: How Agile Masters Deliver Great Software*. The Pragmatic Bookshelf, Raleigh, NC, 2010.

[RW11] Nick Rozanski and Eoin Woods. *Software Systems Architecture: Working With Stakeholders Using Viewpoints and Perspectives*. Addison-Wesley, Boston, MA, Second edition, 2011.

[Sim96] Herbert Simon. *The Sciences of the Artificial*. MIT Press, Cambridge, MA, Third edition, 1996.

[TA05] Jeff Tyree and Art Akerman. Architecture Decisions: Demystifying Architecture. *IEEE Software*. 22[2]:19–27, 2005, March/April.

[VAH12] Uwe Van Heesch, Paris Avgerioum, and Rich Hilliard. A documentation framework for architecture decisions. *Journal of Systems and Software*. 85[4]:795–820, 2012, April, December.

[WFD16] Jonathan Wilmot, Lorraine Fesq, and Dan Dvorak. Quality attributes for mission flight software: A reference for architects. *IEEE Aerospace Conference*. 5–12, 2016, March.

[WNA13] Michael Waterman, James Noble, and George Allen. The effect of complexity and value on architecture planning in agile software development. *Agile Processes in Software Engineering and Extreme Programming (XP2013)*. 2013, May.

[Zwe13] Thomas D. Zweifel. *Culture Clash 2: Managing the Global High Performance Team*. SelectBooks, New York, NY, 2013.

Index

B

C

D